



UNIVERSITÀ DEGLI STUDI DI MESSINA

DIPARTIMENTO DI SCIENZE MATEMATICHE E
INFORMATICHE, SCIENZE FISICHE E SCIENZE DELLA
TERRA

Corso di Laurea triennale in Informatica L-31

**RETI E SISTEMI DISTRIBUITI -
MODULO B**

Progetto finale - Server FTP concorrente in C

Professore
Roberto Marino

Studentessa
Aurora Fabio 558520

Anno Accademico: 2024-2025

Contents

Contents

1	Introduzione	1
2	Definizione del problema	2
3	Metodologia	3
4	Presentazione dei risultati	19
4.1	Funzionalità di base	19
4.2	Test di carico	19
5	Conclusioni	22

1 Introduzione

Per il progetto finale, ho scelto la traccia 2, il cui obiettivo è l'implementazione di un server FTP concorrente in C, che si basa su un'architettura client-server.

Il protocollo **FTP** (File Transfer Protocol), definito nella **RFC 959** è progettato per il trasferimento efficiente e affidabile di file tra host su una rete. L'architettura su cui si basa effettua una separazione tra il canale di controllo e quello dei dati: la **connessione di controllo** (porta 21) è dedicata allo scambio di comandi e risposte, mentre la **connessione dati** (porta 20 in modalità attiva) è dedicata esclusivamente al trasferimento fisico dei file.

Il server FTP è l'entità centrale che gestisce le richieste effettuate dai client. Per garantire la comunicazione, il server si interfaccia con il protocollo di controllo (**Telnet**) e il protocollo di trasporto (**TCP**). Per supportare più client simultaneamente, il server implementa la concorrenza tramite processi, permettendo la gestione di più connessioni parallele attraverso la creazione di un nuovo processo figlio per ogni client connesso.

L'obiettivo di questo progetto è la realizzazione di un server FTP che supporti le seguenti operazioni in modalità anonima:

- CWD per cambiare la directory di lavoro
- LIST per visualizzare l'elenco dei file presenti nella directory corrente
- RETR per scaricare i file dal server
- STOR per caricare i file sul server
- QUIT per terminare la sessione di lavoro

2 Definizione del problema

Il problema principale consiste nella costruzione di un'architettura robusta che rispetti il modello a doppia connessione e gestisca in sicurezza le sessioni multiple simultanee evitando la presenza di colli di bottiglia.

Il protocollo FTP adotta un **paradigma out-of-band**, imponendo una netta separazione logica e fisica tra il flusso di controllo e quello dei dati. Questa architettura richiede al server di mantenere sempre attiva una connessione TCP persistente per lo scambio dei comandi e, contemporaneamente, di orchestrare parallelamente l'apertura e la chiusura on-demand di connessioni dati effimere per ogni richiesta di trasferimento.

Il server deve essere in grado di interpretare correttamente le stringhe di testo inviate dal client sul canale di controllo isolando i comandi dai relativi argomenti e ad ogni richiesta deve eseguire una risposta formattata secondo lo standard (codice a tre cifre e messaggio di testo).

Il server deve essere sempre disponibile ad accettare nuovi client; quando un client sta trasferendo un file di grandi dimensioni, non deve bloccare il server o causare la starvation per gli altri utenti.

3 Metodologia

L'implementazione di questo progetto, ha richiesto un server in ambiente POSIX e l'uso del linguaggio C e delle API dei socket.

```
1 #ifndef COMMON_H
2 #define COMMON_H
3
4 #include <stdio.h>
5 #include <string.h>
6 #include <stdlib.h>
7 #include <sys/socket.h>
8 #include <netinet/in.h>
9 #include <arpa/inet.h>
10 #include <unistd.h>
11 #include <signal.h>
12 #include <sys/wait.h>
13 #include <dirent.h>
14 #include <sys/stat.h>
15 #include <time.h>
16 #include <sys/mman.h>
17 #include <semaphore.h>
18 #include <sys/select.h>
19
20 #define BUFFER_SIZE 1024
21 #define PORT 21
22 #define ROOT_DIR "./ftp_root"
23
24 #endif
```

Listing 1: common.h

```
1 // server FTP concorrente - accept, loop, fork
2 #include "common.h"
3 #include "client.h"
4
5 int *active_clients; // clienti attivi nella memoria
                      // condivisa
6 #define MAX_CLIENTS 20 // limite massimo di connessioni
                      // simultanee
7
8 int main()
9 {
10     int server_fd, new_socket;
11     struct sockaddr_in address;
12     int addrlen = sizeof(address);
13     sem_t *sem; // semaforo per proteggere active_clients
                  // da race condition
14
15     // creazione del socket
```

```
16     if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0)
17     {
18         perror("Creazione del socket fallita");
19         exit(EXIT_FAILURE);
20     }
21
22     // permette il riavvio immediato del server senza
23     // aspettare che la porta di liberi
24     int opt = 1;
25     setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt,
26     sizeof(opt));
27
28     // alloca active_clients nella memoria condivisa con mmap
29     active_clients = mmap(NULL, sizeof(int), PROT_READ |
30     PROT_WRITE, MAP_SHARED | MAP_ANONYMOUS, -1, 0);
31     *active_clients = 0;
32
33     // alloca il semaforo nella memoria condivisa per la
34     // sincronizzazione tra processi
35     sem = mmap(NULL, sizeof(sem_t), PROT_READ | PROT_WRITE,
36     MAP_SHARED | MAP_ANONYMOUS, -1, 0);
37     sem_init(sem, 1, 1);
38
39     // configurazione dell'indirizzo
40     address.sin_family = AF_INET;
41     address.sin_addr.s_addr = INADDR_ANY;
42     address.sin_port = htons(PORT);
43
44     // binding del socket
45     if (bind(server_fd, (struct sockaddr *)&address, sizeof(
46     address)) < 0)
47     {
48         perror("bind failed");
49         close(server_fd);
50         exit(EXIT_FAILURE);
51     }
52
53     // in ascolto per connessioni in entrata
54     if (listen(server_fd, 50) < 0)
55     {
56         perror("listen");
57         close(server_fd);
58         exit(EXIT_FAILURE);
59     }
60
61     // ignora SIGCHLD, il kernel pulisce automaticamente i
62     // figli terminati evitando la creazione di processi zombie
63     signal(SIGCHLD, SIG_IGN);
64     while (1)
```

```
58     {
59         /// accettazione di una nuova connessione
60         if ((new_socket = accept(server_fd, (struct sockaddr
61 *)&address, (socklen_t *)&addrlen)) < 0)
62         {
63             perror("accept");
64             close(server_fd);
65             continue;
66         }
67
68         // lettura protetta dal semaforo per evitare race
69         condition
70         sem_wait(sem);
71         int clients = *active_clients;
72         sem_post(sem);
73
74         // se il limite delle connessioni viene superato, la
75         connessione viene rifiutata con il codice FTP 421
76         if (clients >= MAX_CLIENTS)
77         {
78             send(new_socket, "421 Massimo clients consentiti\
79 r\n", 32, 0);
80             close(new_socket);
81             continue;
82         }
83
84         // fork crea un processo figlio per la gestione del
85         client in parallelo
86         pid_t pid = fork();
87         if (pid < 0)
88         {
89             perror("fork");
90             close(new_socket);
91             continue;
92         }
93
94         if (pid == 0)
95         {
96             // processo figlio
97             close(server_fd); // il figlio non accetta nuove
98             connessioni
99
100             sem_wait(sem); // acquisisce il lock
101             (*active_clients)++; // incrementa il contatore
102             sem_post(sem); // rilascia il lock
103
104             handle_session(new_socket); // il figlio gestisce
105             la sessione FTP del client
```

```
100         sem_wait(sem); // acquisisce il lock
101         (*active_clients)--; // decrementa il contatore
102         sem_post(sem); // rilascia il lock
103
104         exit(EXIT_SUCCESS);
105     }
106     // processo padre
107     close(new_socket);
108 }
109
110 return 0;
111 }
```

Listing 2: main.c

Inizialmente viene creato il socket TCP dedicato al canale di controllo. Ho applicato l'opzione `SO_REUSEADDR` tramite la funzione `setsockopt()` per impedire che la porta 21 rimanga bloccata allo stato `TIME_WAIT` dopo la chiusura del server. Successivamente, il socket viene associato all'indirizzo e alla porta tramite `bind()` e messo in ascolto con `listen()` impostando un massimo di 50 connessioni simultanee.

Poiché l'architettura si basa sui processi che creano dei figli indipendenti con `fork()`, le variabili globali non sarebbero state condivise quindi ho allocato la variabile contatore `active_clients` interna ad un'area di memoria condivisa usando la system call `mmap()` con i flag `MAP_SHARED | MAP_ANONYMOUS`. Per prevenire le **race condition** causate dall'accesso concorrente, ho allocato un semaforo POSIX `sem_t` e prima di leggere/modificare `active_clients`, ogni processo deve acquisire il lock `sem_wait()` e rilasciarlo subito dopo `sem_post()`.

Per evitare la creazione di processi **zombie**, il processo padre deve ignorare il segnale `SIGCHLD` e lo fa grazie alla funzione `SIGNAL(SIGCHLD, SIG_IGN)`; di conseguenza sarà il kernel a ripulire e rimuovere dalla tabella dei processi i figli che hanno terminato l'esecuzione.

Nel loop principale `while(1)`, il server si mette in attesa bloccante tramite `accept()`. All'arrivo di una nuova richiesta:

- il server acquisisce il semaforo per leggere il numero di client attivi. Se il limite è stato raggiunto, rifiuta la connessione, invia il codice d'errore 421 Massimo clients consentiti e chiude il socket.
- se c'è disponibilità, il server esegue la `fork()`

- il processo **figlio** chiude il file descriptor in attesa, acquisisce il semaforo, incrementa in sicurezza il contatore dei client attivi e chiama la funzione `handle_session()`. Al termine della sessione, il figlio decrementa il contatore e termina l'esecuzione.
- il processo **padre** chiude il file descriptor del client accettato e ricomincia il ciclo, tornando in attesa per i nuovi client.

```
1 // gestione comandi FTP
2 #include "common.h"
3 #include "connection.h"
4 #include "client.h"
5
6 void cmd_log(FILE *log_file, const char *cmd)
7 {
8     time_t t = time(NULL);
9     struct tm *tm = localtime(&t);
10    fprintf(log_file, "[%02d:%02d:%02d] %s\n", tm->tm_hour,
11            tm->tm_min, tm->tm_sec, cmd);
12    fflush(log_file); // forza la scrittura immediata su
13    disco
14 }
15
16 // LIST invia la lista dei file della directory sulla data
17 // connection
18 void cmd_list(int client_fd, int data_fd, const char *cwd,
19              FILE *log_file)
20 {
21     DIR *dir;
22     struct dirent *entry;
23     char buffer[1024];
24
25     ftp_reply(client_fd, log_file, 150, "Connessione aperta
26     con successo");
27
28     // apre la directory corrente
29     dir = opendir(cwd);
30     if (dir == NULL)
31     {
32         ftp_reply(client_fd, log_file, 550, "Directory non
33         trovata");
34         close(data_fd);
35         return;
36     }
37
38     // invia ogni entry sulla data connection
39     while ((entry = readdir(dir)) != NULL)
40     {
```

```
35     snprintf(buffer, sizeof(buffer), "%s\r\n", entry->
36     d_name);
37     send(data_fd, buffer, strlen(buffer), 0);
38 }
39     closedir(dir);
40     close(data_fd);
41     ftp_reply(client_fd, log_file, 226, "Trasferimento
42     completo");
43 }
44 // RETR scarica un file dal server al client sulla data
45 // connection
46 void cmd_retr(int client_fd, int data_fd, const char *cwd,
47     const char *filename, FILE *log_file)
48 {
49     char path[1024];
50     char buffer[1024];
51     FILE *file;
52     int n;
53
54     // costruzione path assoluto con directory corrente e
55     // file
56     snprintf(path, sizeof(path), "%s/%s", cwd, filename);
57
58     // apertura in modalità binaria di lettura per supportare
59     // ogni file
60     file = fopen(path, "rb");
61     if (file == NULL)
62     {
63         ftp_reply(client_fd, log_file, 550, "File non trovato
64         ");
65         close(data_fd);
66         return;
67     }
68
69     ftp_reply(client_fd, log_file, 150, "Invio file");
70
71     // legge il file a blocchi (1024 byte) e lo invia sulla
72     // data connection
73     while ((n = fread(buffer, 1, sizeof(buffer), file)) > 0)
74     {
75         send(data_fd, buffer, n, 0);
76     }
77
78     fclose(file);
79     close(data_fd);
80     ftp_reply(client_fd, log_file, 226, "Trasferimento file
81     avvenuto con successo");
```

```
75 }
76
77 // STOR carica un file dal client al server sulla data
    connection
78 void cmd_stor(int client_fd, int data_fd, const char *cwd,
    const char *filename, FILE *log_file)
79 {
80     char path[1024];
81     char buffer[1024];
82     FILE *file;
83     int n;
84
85     snprintf(path, sizeof(path), "%s/%s", cwd, filename);
86
87     ftp_reply(client_fd, log_file, 150, "Pronto alla
    ricezione");
88
89     // apertura in modalità binaria di scrittura per
    supportare ogni file
90     file = fopen(path, "wb");
91     if (file == NULL)
92     {
93         ftp_reply(client_fd, log_file, 550, "Errore nella
    creazione del file di scrittura");
94         close(data_fd);
95         return;
96     }
97
98     // lettura dalla data connection e scrittura sul file a
    blocchi (1024 byte)
99     while ((n = recv(data_fd, buffer, sizeof(buffer), 0)) >
    0)
100     {
101         fwrite(buffer, 1, n, file);
102     }
103
104     fclose(file);
105     close(data_fd);
106     ftp_reply(client_fd, log_file, 226, "Ricezione del file
    con successo");
107 }
108
109 // CWD cambia la directory di lavoro del processo figlio
110 void cmd_cwd(int client_fd, char *cwd, const char *arg, FILE
    *log_file)
111 {
112     char new_path[1024];
113
114     // se il path inizia con / è assoluto, altrimenti è
```

```
relativo alla directory corrente
115     if (arg[0] == '/')
116     {
117         snprintf(new_path, sizeof(new_path), "%s", arg);
118     }
119     else
120     {
121         snprintf(new_path, sizeof(new_path), "%s/%s", cwd,
arg);
122     }
123
124     // cambia la directory del processo figlio
125     if (chdir(new_path) < 0)
126     {
127         ftp_reply(client_fd, log_file, 550, "Directory non
trovata");
128         return;
129     }
130
131     // aggiorna cwd con il percorso pulito per i comandi
successivi
132     if (getcwd(cwd, 1024) == NULL)
133     {
134         perror("Errore getcwd");
135     }
136
137     ftp_reply(client_fd, log_file, 250, "Cambio Directory
effettuato con successo");
138 }
139
140 // invia risposta FTP formattata
141 void ftp_reply(int fd, FILE *log_file, int code, const char *
msg)
142 {
143     char buffer[BUFFER_SIZE];
144     snprintf(buffer, sizeof(buffer), "%d %s\r\n", code, msg);
145     cmd_log(log_file, buffer);
146     send(fd, buffer, strlen(buffer), 0);
147 }
148
149 // gestisce la sessione FTP completa di un singolo client e
viene chiamata dal processo figlio dopo il fork()
150 void handle_session(int client_fd)
151 {
152     char buffer[BUFFER_SIZE];
153     int logged_in = 0; // flag per l'
autenticazione del client
154     char cwd[1024] = "./ftp_root"; // directory di lavoro
iniziale
```

```
155     int data_fd = -1;                // fd sulla data
connection
156     FILE *log_file = fopen("/tmp/server.log", "a");
157
158     ftp_reply(client_fd, log_file, 220, "Benvenuto al server
FTP di Aurora Fabio");
159
160     while (1)
161     {
162         // select() con timeout di 30 secondi: evita che i
client inattivi occupino risorse in modo indefinito
163         fd_set read_fds;
164         struct timeval timeout;
165         FD_ZERO(&read_fds);
166         FD_SET(client_fd, &read_fds);
167         timeout.tv_sec = 30;
168         timeout.tv_usec = 0;
169         int ready = select(client_fd + 1, &read_fds, NULL,
NULL, &timeout);
170         if (ready == 0)
171         {
172             // timeout scaduto e disconnessione del client
173             ftp_reply(client_fd, log_file, 421, "Client
disconnesso per timeout della connessione");
174             break;
175         }
176         if (ready < 0)
177         {
178             perror("Errore select");
179             break;
180         }
181
182         // lettura comando inviato dal client
183         memset(buffer, 0, sizeof(buffer));
184         if (recv(client_fd, buffer, sizeof(buffer) - 1, 0) <=
0)
185         {
186             break;
187         }
188
189         // rimozione di \r\n per confronto con il comando
190         buffer[strcspn(buffer, "\r\n")] = '\0';
191
192         // gestione comandi FTP
193         if (strncmp(buffer, "USER", 4) == 0)
194         {
195             // accettazione di qualsiasi utente (anonimo)
196             cmd_log(log_file, buffer);
197             ftp_reply(client_fd, log_file, 331, "Password
```

```

    richiesta");
198     }
199     else if (strncmp(buffer, "PASS", 4) == 0)
200     {
201         // accetta qualsiasi password (anonimo)
202         logged_in = 1;
203         cmd_log(log_file, buffer);
204         ftp_reply(client_fd, log_file, 230, "Login
effettuato");
205     }
206     else if (strncmp(buffer, "PASV", 4) == 0)
207     {
208         // apertura della data connection in modalità
passiva: il server apre una porta casuale e attende il
client
209         cmd_log(log_file, buffer);
210         data_fd = pasv_open(client_fd);
211     }
212
213     else if (strncmp(buffer, "QUIT", 4) == 0)
214     {
215         // uscita del client
216         cmd_log(log_file, buffer);
217         ftp_reply(client_fd, log_file, 221, "Arrivederci"
);
218         break;
219     }
220     else if (!logged_in)
221     {
222         // blocco di ogni comando se il client non ha
effettuato l'accesso
223         cmd_log(log_file, buffer);
224         ftp_reply(client_fd, log_file, 530, "Effettua l'
accesso");
225     }
226
227     else if (strncmp(buffer, "CWD", 3) == 0)
228     {
229         cmd_log(log_file, buffer);
230         cmd_cwd(client_fd, cwd, buffer + 4, log_file);
231     }
232     else if (strncmp(buffer, "LIST", 4) == 0)
233     {
234         cmd_log(log_file, buffer);
235         cmd_list(client_fd, data_fd, cwd, log_file);
236         // reset, per il prossimo comando bisogna
applicare un nuovo PASV
237         data_fd = -1;
238     }

```

```
239         else if (strncmp(buffer, "RETR", 4) == 0)
240         {
241             cmd_log(log_file, buffer);
242             cmd_retr(client_fd, data_fd, cwd, buffer + 5,
log_file);
243             data_fd = -1;
244         }
245         else if (strncmp(buffer, "STOR", 4) == 0)
246         {
247             cmd_log(log_file, buffer);
248             cmd_stor(client_fd, data_fd, cwd, buffer + 5,
log_file);
249             data_fd = -1;
250         }
251         else if (strncmp(buffer, "PWD", 3) == 0)
252         {
253             cmd_log(log_file, buffer);
254             char pwd_reply[1026];
255             snprintf(pwd_reply, sizeof(pwd_reply), "\"%s\"",
cwd);
256             ftp_reply(client_fd, log_file, 257, pwd_reply);
257         }
258
259         else if (strncmp(buffer, "TYPE", 4) == 0)
260         {
261             // accettazione di valori binari e ASCII
262             cmd_log(log_file, buffer);
263             ftp_reply(client_fd, log_file, 200, "Ok");
264         }
265         else if (strncmp(buffer, "SIZE", 4) == 0)
266         {
267             // per non bloccare il client
268             cmd_log(log_file, buffer);
269             ftp_reply(client_fd, log_file, 502, "Ok");
270         }
271         else
272         {
273             cmd_log(log_file, buffer);
274             ftp_reply(client_fd, log_file, 502, "Comando
sconosciuto!");
275         }
276     }
277
278     fclose(log_file);
279     close(client_fd);
280 }
```

Listing 3: client.c

```
1 #ifndef CLIENT_H
2 #define CLIENT_H
3
4 void cmd_log(FILE *log_file, const char *cmd);
5 void cmd_list(int clientfd, int data_fd, const char *cwd,
6 FILE *log_file);
7 void cmd_retr(int client_fd, int data_fd, const char *cwd,
8 const char *filename, FILE *log_file);
9 void cmd_stor(int client_fd, int data_fd, const char *cwd,
10 const char *filename, FILE *log_file);
11 void ftp_reply(int fd, FILE *log_file, int code, const char *
12 msg);
13 void cmd_cwd(int client_fd, char *cwd, const char *arg, FILE
14 *log_file);
15 void handle_session(int client_fd);
16
17 #endif
```

Listing 4: client.h

Dopo che il processo figlio prende in carico il client, viene eseguita la funzione `handle_session()`. Per evitare che un client inattivo tenga il processo figlio all'infinito, non ho usato la system call bloccante `recv()`, ma ho implementato un meccanismo di attesa con timeout tramite la system call `select()`, configurandola a 30 secondi. Se il client non invia alcun comando entro quel tempo, la `select()` scade, il server invia un avviso di disconnessione 421 Client disconnesso per timeout della connessione e termina il processo, liberando le risorse.

Per il server anonimo, ho simulato un sistema di autenticazione: il server accetta qualsiasi stringa inerente a `USER` e `PASS` impostando il flag `logged_in=1`. Tuttavia se il client non si *autentica* prima di effettuare una richiesta, il server la blocca e restituisce l'errore 530 Effettua l'accesso.

Il server tiene traccia della directory corrente tramite la stringa `cwd`. Il comando `PWD` restituisce la directory tra virgolette, mentre `CWD` analizza l'argomento trattandolo come percorso assoluto o relativo: successivamente, la system call `chdir()` cambia effettivamente la directory di lavoro del processo figlio e, subito dopo, la funzione `getcwd()` viene utilizzata per estrarre il percorso assoluto pulito e aggiornare in sicurezza la stringa `cwd`.

I comandi FTP che richiedono un canale dati sono:

- **LIST** per scorrere il contenuto della directory tramite le funzioni `opendir()` e `readdir()`
- **RETR** costruisce ed apre il percorso assoluto del file in modalità di lettura binaria `rb` tramite la funzione `fopen()`. Successivamente, legge il file a blocchi da 1024 byte tramite `fread()` e lo invia al client tramite `send()`
- **STOR** apre o crea il file di destinazione in modalità di scrittura binaria `wb`. Successivamente, il server entra in un ciclo di `recv()` sul canale dati, legge i blocchi in arrivo e li scrive su disco con `fwrite()`

Al termine di ognuna delle tre operazioni, il file descriptor dei dati `data_fd` viene chiuso con `close()` e resettato a `-1`. Per il comando successivo, il client deve richiedere una nuova connessione passiva.

Inoltre, per la gestione dei log e delle risposte, ho implementato e utilizzato le seguenti funzioni:

- `cmd_log()` per registrare ogni comando in ingresso in un file di log locale `/tmp/server.log`, avvalendomi della funzione di libreria `fflush()` per garantire che i log siano scritti fisicamente sul disco in tempo reale.
- `ftp_reply()` come funzione wrapper che formatta il codice a tre cifre, invia le risposte standard al client e registra automaticamente anche le risposte del server nel file di log, permettendo il tracciamento completo dell'intera conversazione.

Infine, per evitare che alcuni client si blocchino aspettando risposte a comandi non essenziali, ho gestito passivamente **TYPE** e **SIZE** restituendo un codice di successo senza interrompere la connessione.

```
1 // data connection FTP in modalità passiva PASV
2 #include "common.h"
3 #include "connection.h"
4
5 // apertura porta passiva, notifica del client e restituzione
  del data_fd
6 int pasv_open(int client_fd)
7 {
8     int pasv_fd;
9     struct sockaddr_in addr;
10    socklen_t addr_len = sizeof(addr);
11
12    // creazione socket TCO per la data connection
13    pasv_fd = socket(AF_INET, SOCK_STREAM, 0);
```

```
14     if (pasv_fd < 0)
15     {
16         perror("Errore creazione socket pasv_fd");
17         exit(EXIT_FAILURE);
18     }
19
20     // bind su porta casuale per conflitti con più client o
21     // porte già in uso
22     memset(&addr, 0, sizeof(addr));
23     addr.sin_family = AF_INET;
24     addr.sin_addr.s_addr = INADDR_ANY;
25     addr.sin_port = 0;
26
27     if (bind(pasv_fd, (struct sockaddr *)&addr, sizeof(addr))
28         < 0)
29     {
30         perror("Errore bind pas_fd");
31         exit(EXIT_FAILURE);
32     }
33
34     // ascolto solo di 1 client
35     if (listen(pasv_fd, 1) < 0)
36     {
37         perror("Errore listen pas_fd");
38         exit(EXIT_FAILURE);
39     }
40
41     // porta scelta dal sistema operativo
42     getsockname(pasv_fd, (struct sockaddr *)&addr, &addr_len)
43     ;
44
45     int port = ntohs(addr.sin_port);
46     int p1 = port / 256;
47     int p2 = port % 256;
48
49     // invio notifica al client che dovrà connettersi con
50     // essi sulla data connection
51     char reply[64];
52     snprintf(reply, sizeof(reply), "227 Accesso modalità
53     passiva (127,0,0,1,%d,%d)\r\n", p1, p2);
54     send(client_fd, reply, strlen(reply), 0);
55
56     // attesa connessione client
57     int data_fd = accept(pasv_fd, NULL, NULL);
58     close(pasv_fd);
59
60     return data_fd;
61 }
```

Listing 5: connection.c

```
1 #ifndef CONNECTION_H
2 #define CONNECTION_H
3
4 int pasv_open(int client_fd);
5
6 #endif
```

Listing 6: connection.h

Ho implementato la **modalità passiva (PASV)** per il canale secondario usato per l'effettivo trasferimento dei file tramite la funzione `pasv_open()`. Ciò rende il server più affidabile poiché delega il client all'instaurazione della connessione dati bypassando i problemi di firewall.

Quando il server riceve il comando PASV, viene creato `pasv_fd`, un nuovo socket TCP. Per evitare che i due processi figli aprano un canale dati sulla stessa porta, ho configurato il campo della porta a 0. La funzione `bind()` assegna automaticamente una porta casuale attualmente libera. Poiché la porta è stata scelta dal kernel, ho chiamato `getsockname()` che definisce la struttura `addr` con le informazioni necessarie mentre la funzione `ntohs()` converte il numero di porta dal formato rete al formato host.

Per la risposta a PASV, la RFC 959 impone che l'indirizzo IP e la porta devono essere espressi come sei numeri separati da virgole (`h1,h2,h3,h4,p1,p2`). I primi quattro rappresentano l'indirizzo IP (`127,0,0,1`) e i restanti due la porta: `p1 = PORT / 256`, cioè il bit più significativo; `p2 = PORT % 256`, cioè il bit meno significativo.

Ho impostato la `listen()` con un backlog pari a 1 perché mi aspetto una sola connessione per ogni comando PASV, il processo figlio si mette in attesa bloccante con `accept()`. Quando il client si connette al server, la `accept()` restituisce un nuovo file descriptor `data_fd`. Successivamente, per liberare risorse e impedire altre connessioni sulla stessa porta, il server esegue `close(pas_fd)` sul socket in ascolto e restituisce `data_fd`.

```
1 CC = gcc
2 CFLAGS = -Wall
3
4 all: main
5
6 main: main.c client.c connection.c
7     $(CC) $(CFLAGS) -o main main.c client.c connection.c -lrt
8
9 clean:
10     rm -f main
```

Listing 7: Makefile

Ho adottato un approccio modulare affinché la logica del programma sia separata in file distinti:

- `main.c` per la configurazione iniziale e l'architettura concorrente
- `client.c` per il parsing e i comandi FTP
- `connection.c` per la gestione del canale dati passivo

Per automatizzare il processo di build, ho disposto un `Makefile` nel quale ho implementato:

- `-Wall` come flag di compilazione per abilitare tutti i warning
- `-lrt` come linking delle librerie POSIX utilizzate per la memoria condivisa e i semafori
- `clean` per rimuovere l'eseguibile `main` generato

4 Presentazione dei risultati

Per validare il corretto funzionamento e l'affidabilità del server FTP, ho condotto diversi test, sia sulle funzionalità di base sia di carico sotto sforzo. Per entrambi i metodi avvio il server tramite il comando `sudo ./server` poiché servono i comandi da amministratore visto che utilizzo la porta 21.

4.1 Funzionalità di base

Ho verificato la conformità del server tramite un client FTP da terminale ed ho eseguito la seguente sequenza operativa per validare tutte le funzionalità richieste:

```
1 ftp> ls                # LIST
2 ftp> cd testing        # CWD
3 ftp> ls
4 ftp> cd ..
5 ftp> get test.txt      # RETR
6 ftp> put test.txt      # STOR
7 ftp> bye
```

Tramite i comandi `ls` e `cd`, ho verificato che il server mostri correttamente il contenuto della cartella ed aggiorni la directory di lavoro spostandosi nella directory `testing`, mostrarne il contenuto e tornare indietro in modo coerente. Ad ogni chiamata, la connessione dati passiva si è aperta e chiusa regolarmente.

Successivamente ho eseguito il download (`get`) e l'upload (`put`) del file `test.txt` e l'apertura dei file in modalità `rb` e `wb` ha garantito il corretto trasferimento dei byte.

Infine, tramite il comando `bye` è stata effettuata la disconnessione del processo figlio, confermata dal messaggio e dalla pulizia delle risorse.

4.2 Test di carico

Per verificare l'efficacia dell'architettura multi-processo e il corretto uso dei semafori, ho progettato uno script bash che simula uno scenario di alto traffico

```
1 #!/bin/bash
2
3 # test di carico per lanciare N client simultaneamente e
  verificare la corretta gestione del server che riguarda la
  concorrenza
4
5 # directory radice per get e put
6 cd ~
```

```
7 echo "test" > test.txt
8
9 N=50 # numero di client simultanei
10 for i in $(seq 1 $N); do
11     (
12         # lancio di ogni client in background con & e <<EOF
13         # per lanciare i comandi automaticamente senza interazioni
14         ftp -p -P 21 127.0.0.1 <<EOF
15         anonymous
16         password
17         ls
18         get test.txt
19         put test.txt
20         bye
21     ) &
22 done
23
24 wait # aspetta che tutti i processi in background terminino
25 echo "Test completato con $N client simultanei"
```

Listing 8: test_carico.sh

Lo script lancia N istanze del client FTP. Tramite il flag `-p` è stato forzato l'uso di PASV, cioè della modalità passiva per non generare conflitti a causa dei canali dati già usati.

Avendo impostato `MAX_CLIENTS = 20`, lo script ha permesso di validare la logica della memoria condivisa ed il server ha gestito parallelamente e senza deadlock i trasferimenti delle prime 20 connessioni. Le restanti 30 richieste vengono bloccate dal processo padre, che ne rifiuta l'accesso prevenendo efficacemente il sovraccarico del sistema.

Il comando `SIG_IGN` su `SIGCHLD` ha garantito l'assenza dei processi zombie.

Infine, grazie a **Wireshark** ho potuto ispezionare i pacchetti in transito durante le sessioni. Catturando il traffico tramite `lo`, ho verificato visivamente la corretta separazione dei due canali. Sul canale di controllo (porta 21) ho constatato il corretto scambio in chiaro delle stringhe di testo e dei codici a tre cifre. In particolare, Wireshark mi ha permesso di tracciare il momento in cui il server invia la porta calcolata con la risposta 227 **Accesso modalità passiva** e di osservare il successivo **Three-Way Handshake** TCP che il client instaura verso quella porta per aprire il canale dati.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	74	59386 → 21 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 SACK_PERM TSval=1925765854 TSecr=...
2	0.000029404	127.0.0.1	127.0.0.1	TCP	74	21 → 59386 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=192576...
3	0.000053010	127.0.0.1	127.0.0.1	TCP	66	59386 → 21 [ACK] Seq=1 Ack=1 Win=130992 Len=0 TSval=1925765854 TSecr=1925765854
4	0.000022637	127.0.0.1	127.0.0.1	FTP	111	Response: 220 Benvenuto al server FTP di Aurora Fabio
5	0.000047990	127.0.0.1	127.0.0.1	TCP	66	59386 → 21 [ACK] Seq=1 Ack=46 Win=130948 Len=0 TSval=1925765855 TSecr=1925765855
11	11.519254672	127.0.0.1	127.0.0.1	FTP	82	Request: USER anonymous
12	11.519291758	127.0.0.1	127.0.0.1	TCP	66	21 → 59386 [ACK] Seq=46 Ack=17 Win=65536 Len=0 TSval=1925777373 TSecr=1925777373
13	11.519473095	127.0.0.1	127.0.0.1	FTP	99	Response: 331 Password richiesta
14	11.519450804	127.0.0.1	127.0.0.1	TCP	66	59386 → 21 [ACK] Seq=17 Ack=70 Win=130944 Len=0 TSval=1925777373 TSecr=1925777373
15	12.282756979	127.0.0.1	127.0.0.1	FTP	73	Request: PASS
16	12.282954560	127.0.0.1	127.0.0.1	FTP	88	Response: 230 Login effettuato
17	12.282979513	127.0.0.1	127.0.0.1	TCP	66	59386 → 21 [ACK] Seq=24 Ack=92 Win=130948 Len=0 TSval=1925778137 TSecr=1925778137
18	12.283106675	127.0.0.1	127.0.0.1	FTP	72	Request: SYST
19	12.283215767	127.0.0.1	127.0.0.1	FTP	92	Response: 502 Comando sconosciuto!
20	12.283315012	127.0.0.1	127.0.0.1	FTP	72	Request: FEAT
21	12.283434232	127.0.0.1	127.0.0.1	FTP	92	Response: 502 Comando sconosciuto!
22	12.323026208	127.0.0.1	127.0.0.1	TCP	66	59386 → 21 [ACK] Seq=36 Ack=144 Win=130968 Len=0 TSval=1925778178 TSecr=1925778137
27	42.313680715	127.0.0.1	127.0.0.1	FTP	120	Response: 421 Client disconnesso per timeout della connessione
28	42.313711725	127.0.0.1	127.0.0.1	TCP	66	59386 → 21 [ACK] Seq=36 Ack=198 Win=130916 Len=0 TSval=1925808168 TSecr=1925808168
29	42.313759515	127.0.0.1	127.0.0.1	TCP	66	21 → 59386 [FIN, ACK] Seq=198 Ack=36 Win=65536 Len=0 TSval=1925808168 TSecr=192580...
30	42.354677326	127.0.0.1	127.0.0.1	TCP	66	59386 → 21 [ACK] Seq=36 Ack=199 Win=130916 Len=0 TSval=1925808209 TSecr=1925808168

Figure 1: Accesso

No.	Time	Source	Destination	Protc	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	74	41782 → 21 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 SACK_PERM TSval=1925950610 TSecr=0 WS=4
2	0.000048951	127.0.0.1	127.0.0.1	TCP	74	21 → 41782 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=1925950610 TSecr=1925950610 WS=1024
3	0.000075429	127.0.0.1	127.0.0.1	TCP	66	41782 → 21 [ACK] Seq=1 Ack=1 Win=130992 Len=0 TSval=1925950610 TSecr=1925950610
4	0.0000620472	127.0.0.1	127.0.0.1	FTP	111	Response: 220 Benvenuto al server FTP di Aurora Fabio
5	0.0000646732	127.0.0.1	127.0.0.1	TCP	66	41782 → 21 [ACK] Seq=1 Ack=46 Win=130948 Len=0 TSval=1925950610 TSecr=1925950610
6	5.019126713	127.0.0.1	127.0.0.1	FTP	72	Request: EPSV
7	5.019151494	127.0.0.1	127.0.0.1	TCP	66	21 → 41782 [ACK] Seq=46 Ack=7 Win=65536 Len=0 TSval=1925955629 TSecr=1925955629
8	5.019328555	127.0.0.1	127.0.0.1	FTP	99	Response: 530 Effettua l'accesso
9	5.019349996	127.0.0.1	127.0.0.1	TCP	66	41782 → 21 [ACK] Seq=7 Ack=70 Win=130944 Len=0 TSval=1925955629 TSecr=1925955629
10	5.019473476	127.0.0.1	127.0.0.1	FTP	72	Request: PASV
11	5.019634111	127.0.0.1	127.0.0.1	FTP	116	Response: 227 Accesso modalita passiva (127,0,0,1,128,67)

Figure 2: Accesso modalit  passiva

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	FTP	81	Request: SIZE test.txt
2	0.000174185	127.0.0.1	127.0.0.1	FTP	74	Response: 502 Ok
3	0.000196604	127.0.0.1	127.0.0.1	TCP	66	36672 → 21 [ACK] Seq=16 Ack=9 Win=32740 Len=0 TSval=1926102862 TSecr=1926102862
4	0.000331538	127.0.0.1	127.0.0.1	FTP	72	Request: PASV
5	0.000489449	127.0.0.1	127.0.0.1	FTP	117	Response: 227 Accesso modalit� passiva (127,0,0,1,221,127)
9	0.000927355	127.0.0.1	127.0.0.1	FTP	81	Request: RETR test.txt
10	0.001387541	127.0.0.1	127.0.0.1	FTP	82	Response: 150 Invio file
11	0.001603631	127.0.0.1	127.0.0.1	FTP-DATA	72	FTP Data: 6 bytes (PASV) (RETR test.txt)
16	0.042060505	127.0.0.1	127.0.0.1	TCP	66	36672 → 21 [ACK] Seq=37 Ack=76 Win=32742 Len=0 TSval=1926102904 TSecr=1926102863
17	0.042096614	127.0.0.1	127.0.0.1	FTP	96	Response: 226 Trasferimento completato

Figure 3: FTP-data

5 Conclusioni

Lo sviluppo di questo progetto mi ha permesso la realizzazione di un server FTP concorrente, stabile e conforme alle direttive dello standard **RFC 959**.

Affrontare il paradigma **out-of-band** a doppia connessione ha richiesto l'implementazione della modalità passiva (**PASV**) e la gestione dinamica delle porte effimere a livello del sistema operativo.

Inoltre, la gestione del parallelismo dei processi ha portato alla gestione delle risorse concorrenti, che ho risolto tramite la comunicazione inter-processo (**IPC**) con aree di memoria condivisa (**mmap**) e semafori (**sem_t**). Per prevenire i processi **zombie**, ho istruito il server a ignorare il segnale **SIGCHLD**, mentre per gestire in modo sicuro l'inattività dei client ho utilizzato la **select()**. Ciò ha reso il server robusto.

I possibili sviluppi futuri che migliorerebbero il progetto sono:

- l'uso dei **thread** al posto dei **processi** per le loro prestazioni
- l'implementazione di un'autenticazione reale e non più anonima
- l'ampliamento dei comandi FTP, come l'eliminazione (**DELE**) o la creazione (**MKD**) delle directory